LETMA 2026 Summer School
Machine Learning on Geophysical Data — Labs: Solutions & Expected Results

The answers to the exercises and the numbers every run reproduces

E. Noëlé, Gatti, Lehmann, Millet, Clouteau
AMIAD, CEA, LMPS, Borelli
Contact: `elodie.noele@TODO.example`

This is the companion of the student handout: for each lab it gives the exercise solutions and the expected results. Every number below was produced by executing the `*_solution` notebooks end to end on a MIG slice (`1g.20gb`) of an H100 on the Hedy cluster; the executed notebooks contain every figure and animation, so any of these results can be shown live without re-running a cell.

Lab 1 — FNO for the 1-D Burgers equation

Exercise F1 (spectral convolution)

```
def forward(self, x):
    n = x.shape[-1]
    xh = torch.fft.rfft(x, dim=-1, norm='ortho') # (B, C, N//2+1)
    m = min(self.modes, xh.shape[-1]) # truncation (H7)
    out = torch.einsum('bik,iok->bok', xh[..., :m], self.weight[..., :m])
    return torch.fft.irfft(out, n=n, norm='ortho') # back to physical space
```

Answer to the question. Keeping $m = 16$ modes loses almost nothing because the target fields' energy is concentrated in the first Fourier modes (H7, checked on data): $m = 16$ keeps 99.8% of the mean spectral energy (19 modes reach 99.9%) and reconstructs a target to $\approx 3\%$ relative error — still small next to the $\approx 8\%$ test error.

Question: where does the computation happen?

Lift $a(x) \oplus x$ to $w = 32$ channels (P), then four blocks $v_{\ell+1} = \mathrm{GELU}(K_\ell v_\ell + W_\ell v_\ell)$, then Q (width 128) back to one channel. **Answer:** the computation happens in the learned 32-channel latent space — the blocks never see u again after the lift.

Expected results

validation rel-L^2 (60 epochs, AdamW+cosine)	≈ 0.083
test rel-L^2 (untouched split)	mean 0.076, median 0.075, worst 0.179
zero-shot super-resolution N=64 $\to$ N=256	0.073 (no retraining)
optimizer race to L^*	Adam **79 steps**, SGD+momentum **604 steps**, plain SGD still far above L^* after 6000 steps
two-seed minima	two distinct minima, three Adam runs fall in separate basins
corner plot (best of 20 configs, 25 epochs)	modes 16, width 48, 6 layers, proj 128 $\to$ 0.097

Lab 2 — FFNO for infrasound boundary-layer propagation

Exercise F2 (signal spectrum)

```
def signal_spectrum(trace, dt):
    f = np.fft.rfftfreq(len(trace), d=dt) # frequencies of the rfft bins
    return f, np.fft.rfft(trace) # complex spectrum
```

Answer to the question. The energy of a recorded trace lives inside the band $[0.3, 4]$ Hz propagated by the solver (H4); about 64 of the 129 time Fourier modes carry the whole band — exactly the truncation the factorized operator makes (H13).

Expected results

demo atmosphere	c_{eff} 323.8 $\to$ 361.0 m/s; 10 ground arrivals within 11 km; hodograph arrivals at $t = r/c_{\text{ref}}+$ wavelet delay
training (240 epochs, 200 atmospheres)	validation rel-$L^2 = 0.212$ (145,345 parameters)
test (28 unseen atmospheres)	mean 0.336, worst 0.840
loss family (60 epochs)	rel-L^2 0.276 vs MSE 0.301 (rel-L^2 with aggressive $\lambda = 1$: 0.288)
overfitting demo (8 atmospheres)	train 0.476 vs validation 1.125 (gap $\times 2$)
optimizer race (3 optimizers)	Adam: 1.00 $\to$ 0.199 in 153 steps ($L^* = 0.175$); SGD and SGD+momentum remain near 1.0 after 4000 steps (near-silence initialization)

Lab 3 — GNN for a sensor network

Exercise G1 (message passing)

```
def message(self, h_i, h_j, pos_i, pos_j):
    rel = pos_j - pos_i
    return self.msg(torch.cat([h_i, h_j, rel, rel.norm(dim=-1, keepdim=True)], -1))
def forward(self, h, pos, edge_index):
    return h + self.upd(torch.cat([h, self.propagate(edge_index, h=h, pos=pos)], -1))
```

Exercise G2 (the locator)

```
h = self.encoder(data.x) # one 64-dim state per station
for layer in self.layers: # 4 message-passing layers
    h = layer(h, data.pos, data.edge_index)
pooled = global_mean_pool(h, data.batch) # invariant event code
return self.decoder(pooled) # normalized hypocentre
```

Expected results

smoothing-operator spectrum (demo graph)	$\rho = 0.844$: the slowest non-mean mode keeps 0.51 of its amplitude at depth 4, 0.066 at depth 16
depth experiment (validation error)	1 layer 0.47 2: 0.43 4: 0.43 8: 0.37 16: 0.39
locator (60 epochs)	validation ≈ 0.31 (normalized; $1.0 \sim 20$ km)
epicentral error (50 test events)	median 4.87 km, mean 5.08, 90% 8.23
graph-free MLP baseline	median 6.92 km (pays for every missing station)

Answers to the questions. The aggregation must be permutation-invariant (mean, sum or max): it treats neighbours as an unordered set, whereas concatenating neighbour features in a fixed order would change when stations are relabeled. The GNN survives station dropout because the graph and the masked pooling adapt to any node count; the fixed-size MLP has one slot per station and loses the information of every station that is down. Node states become interchangeable when the Dirichlet energy collapses: one scalar per depth is the diagnostic.

Optional extensions

- **Darcy scale-up:** a 123k-parameter FFNO2d, 200 epochs on 2304 pairs at 96×96: test rel-L^2 0.0083 (MIG, ≈ 9 min).
- **Labs 1 and 3 in JAX** (Equinox + Optax): FNO val 0.0969, test 0.0867, zero-shot at N=256: 0.0889; GNN epicentral median 3.05 km vs MLP 7.04 km. Same physics, different framework — the notebooks need neither torch nor PyG (data shipped as arrays).

How to re-run everything

Connect and work on Hedy (one SSH hop)

```
ssh -Y hedy # the single hop (-Y: X11 forwarding)

# either use the pre-built shared environment (nothing to install):
source /home/projects/m26266/letma2026-ml-labs/env/fno-gnn/bin/activate

# or install the requirements into your own venv:
python3 -m venv ~/envs/fno-gnn && source ~/envs/fno-gnn/bin/activate
pip install -r /home/projects/m26266/letma2026-ml-labs/requirements.txt

# copy the labs into YOUR home folder on Hedy (env/ excluded: the venvs are 5+ GB):
rsync -a --exclude env /home/projects/m26266/letma2026-ml-labs/ ~/letma2026-ml-labs/

# GPU session on a MIG slice: ONE command allocates the slice, forwards X11
# and drops you into an interactive shell ON the GPU node:
salloc \
    --account=m26266 \
    --partition=gpu \
    --reservation=test_mig_letma \
    --nodelist=m-gpu[07-08] \
    --gres=gpu:nvidia_h100_80gb_hbm3_1g.20gb:1 \
    --nodes=1 \
    --ntasks=1 \
    --cpus-per-task=8 \
    --time=01:00:00 \
    bash -c '
        NODE=$(scontrol show hostnames "$SLURM_JOB_NODELIST" | head -n 1)
        echo "Allocated job: $SLURM_JOB_ID on $NODE"
        exec ssh -Y -C -t "$NODE" \
            "srun --jobid=$SLURM_JOB_ID --overlap --ntasks=1 --pty bash -i"
    '
# release afterwards with: scancel <jobid>

# on the GPU node, first CHECK YOUR SLICE - one MIG device with ~20 GB:
nvidia-smi
# MIG devices:
# GPU GI CI MIG | Shared Memory-Usage
```

```
# 0 3 0 0 | 9MiB / 20096MiB <- your 1g.20gb slice
# (if the MIG section is absent or shows another size, exit and resubmit the salloc)

# open a notebook with Euporie, the notebook TUI (Hedy has no SSH port
# forwarding, so there is no browser interface on the cluster; jupyter lab
# remains the normal interface on the participants' own machines):
cd ~/letma2026-ml-labs/notebooks/labs
euporie-notebook --kernel-name python3 lab_fno_solution.ipynb
```

Launch time: about 5 seconds on a warmed node, 1–2 minutes on a cold one. Everything (euporie, jupyter, torch, numpy, matplotlib, ...) is paged in from the shared filesystem on the first launch of a node; the screen shows `Loading...` / `No Kernel` meanwhile — this is not a freeze. **Before the session, warm up both nodes** (the Linux page cache is shared by all users of a node, so one run makes every participant's launch quick):

```
ssh hedy # no -Y needed here: this submits a batch job, nothing is displayed
sbatch /home/projects/m26266/letma2026-ml-labs/slurm/prewarm_nodes.sbatch # ~3 min, both
     nodes
```

Participants then wait only for the kernel badge `Python 3 (ipykernel)` to appear in the bottom-right status bar. The `Rendering... (61%)` indicator may stay at a partial value: it counts the whole notebook, and cells below the visible area are rendered lazily when you scroll.

Euporie quick reference (verified against the installed version): `Up/Down` (or `j/k`) select cells, `Home/End` first/last, `PgUp/PgDn` ±5; `Enter` enters edit mode, `Esc` leaves it; `Ctrl+Enter` runs the selected cell, `Shift+Enter` runs and selects the next, `Alt+Enter` runs and inserts below; `a/b` insert cells above/below, `d d` deletes, `z` undeletes, `x/c/v` cut/copy/paste, `m/y` set the cell type; `i i` interrupts the kernel, `0 0` restarts it; `Ctrl+S` saves, `Ctrl+O` opens, `Ctrl+Q` quits (prompting to save unsaved changes); `Tab` cycles the focus to the command bar for the rarer actions. Figures render inline (terminal graphics on Kitty/Sixel terminals, character previews elsewhere). The `python3` kernel is the activated venv's interpreter — the validated one. The JAX extensions use the sibling venv `env/jax-hedy` (activate it the same way). To work on a GPU: `ssh -Y hedy`, then run the single `salloc` command of the listing below: it allocates the MIG slice, forwards X11 with `ssh -Y` into your allocated node, and drops you into an interactive shell there via `srun --pty` (which is why `srun --x11` is not needed here).

Headless re-validation of the whole set

On Hedy, from the shared folder, one command re-executes all six lab notebooks plus the extensions headlessly (see `slurm/PROCEDURE_TESTS.md`):

```
sbatch slurm/test_all_mig.sbatch
```

Mandatory MIG directives are already in the script. Total measured time: labs ≈ 5 min, Darcy ≈ 9 min, JAX ≈ 7 min.